

SAE Interventions are Unreliable: Post-Intervention Recovery of Suppressed Behavior

Mingyue Cui Linghui Shen Xingyi Yang*

The Hong Kong Polytechnic University

{ming-yue.cui, ling-hui.shen}@connect.polyu.hk, xingyi.yang@polyu.edu.hk

Abstract

Sparse Autoencoders (SAEs) decompose the residual-stream activations into interpretable features. Recent latent-space defenses increasingly rely on these decompositions, assuming that identified "unsafe" SAE features serve as actionable handles for monitoring and intervention. Under this paradigm, clamping a specific harmful feature is expected to reliably prevent model misbehavior. However, we show that this intervention success may hide a recoverable failure mode: the clamp may block one visible route to a behavior without eliminating the behavior itself. We formulate this vulnerability as post-intervention recovery, a constrained residual-space optimization problem. Starting from the post-intervention residual state, we optimize residual perturbations to recover the pre-intervention behavior while preserving the post-intervention values of the targeted SAE features. Even under a strong threat model where the intervention remains active throughout optimization and generation, recovery remains possible. To rule out the possibility that recovery simply undoes the intervention, we use encoder-orthogonal updates for single-layer interventions and the corresponding feature-map Jacobian in the cross-layer setting. Across TPP, unlearning, IOI, and refusal steering experiments, this stress test reveals recoverable behavior despite successful feature-level intervention. Especially in the safety-critical refusal-steering setting, we achieve a 95.8% recovery rate on valid samples while keeping defended-feature relative drift to 0.131, substantially below suffix-based baselines. A recovery-path attribution analysis further localizes this recovery to the SAE reconstruction residual, the component left unexplained by the SAE. These results expose a gap between feature-level control and behavioral completeness: SAE features can support causal intervention, but controlling them does not guarantee control over the underlying behavior. Code is available at [Mingyue88/sae-post-intervention-recovery](https://github.com/Mingyue88/sae-post-intervention-recovery).

1 Introduction

Representation-level safety methods aim to control language-model behavior before harmful content is produced [24, 11, 21, 17, 25]. Sparse autoencoders (SAEs) make this approach especially attractive by decomposing residual-stream activations into sparse and interpretable features [7, 6, 3, 20, 9, 13]. These features appear to offer concrete handles for analyzing, monitoring, and controlling model behavior [15, 1]. Building on this promise, recent latent-space defenses identify features associated with unsafe or unwanted behaviors and then clamp or suppress those features during inference [16, 23, 8, 10, 19]. Implicit in this paradigm is a strong mechanistic assumption: an identified SAE feature is treated not only as a correlate of the behavior, but as a reliable intervention target whose removal should completely disable the behavior from reappearing.

Despite these successes, this assumption deserves closer examination. Suppressing a targeted SAE feature may block the most salient computational route to a behavior, but it does not necessarily remove the model’s underlying capacity to produce it. The model may instead rely on alternative

directions, downstream layers, or distributed mechanisms that are not captured by the targeted feature set [4, 5, 12, 14]. In such cases, the defense merely changes the route through which the behavior is expressed rather than eliminating the behavior itself. If the suppressed behavior can be fully recovered without reactivating the targeted SAE features, then the intervention has not established a true behavioral bottleneck.

To test this limitation directly, we introduce **post-intervention recovery** as a white-box diagnostic. The goal is not to evade feature detection before a defense is applied [2]. Instead, we begin after the intervention has already been deployed. We assume that the relevant SAE features have been identified and clamped. We then ask a sharper question: *from this post-intervention residual state, can the model’s pre-intervention behavior still be restored?*

To implement this diagnostic, we formulate post-intervention recovery as a *constrained residual-space optimization problem*. Starting from the clamped state, we optimize small residual perturbations to restore the target behavior. To prevent the optimization from simply undoing the clamp and to deeply understand the underlying mechanism, we introduce two technical pillars:

- ▷ **Geometric Constraints via Projected Gradient Descent.** We impose constraints on the update directions to force recovery in the null space of representation, rather than directly reactivating the targeted SAE features. Specifically, for single-layer interventions, we project updates away from the selected SAE encoder directions. For cross-layer interventions, we use feature-map Jacobians to constrain how perturbations affect the features across layers.
- ▷ **Recovery-Path Attribution.** Beyond measuring *whether* recovery occurs, we investigate *where* the recovery happens. By decomposing the recovered residual state, we distinguish whether the target behavior compensates through non-clamped SAE latents or exploits the SAE-unexplained reconstruction residual.

Through our recovery, we find that SAE interventions can be easily restored, as recovery paths still exist even when the relevant behavior is suppressed. At the latent level, TPP on SAEBench [10] shows that encoder-orthogonal recovery preserves a high behavioral recovery rate of 74.9% while sharply restricting targeted-feature reactivation to just 0.002. At the output level, WMDP-Bio unlearning [8] demonstrates that recovery restores 98.9% of strict valid answer-choice flips from the post-intervention state with zero measured activation drift. At the circuit level, IOI [22] shows that the encoder-projected method achieves 100% recovery with only a 0.016 reactivated-feature fraction. Finally, in the safety refusal-steering task [16, 23], recovery reaches 95.8% on strict-valid AdvBench prompts while keeping defended-feature relative drift to 0.131. Notably, our attribution analysis reveals that this recovered malicious behavior is primarily carried by the SAE reconstruction residual rather than by alternative visible SAE features. Together, these results suggest that SAE features can be useful local causal handles without forming complete intervention bottlenecks.

2 Related Work

SAE features as useful but incomplete handles. Sparse autoencoders expose sparse latents that can serve as interpretable handles for editing, steering, and circuit analysis [3, 15]. If intervening on a feature changes behavior, that feature is causally relevant; however, causal relevance does not imply completeness. Work on superposition, SAE geometry, feature absorption, feature hedging, and sparse feature circuits suggests that behaviorally relevant information can be distributed across correlated directions or split across multiple latents [7, 12, 4, 5, 15]. This motivates our question: after a selected SAE feature set is fixed, can the same behavior still be recovered through residual directions outside that set?

From monitor bypass to post-intervention recovery. Latent-space defenses often detect harmful or unwanted behavior in activation space and then suppress it by intervention. OABD studies the monitoring stage, showing that harmful behavior can persist while activations evade probes, SAE-latent monitors, and OOD detectors [2]. We study the later clamp stage: the relevant features have already been selected, the intervention remains active, and we ask whether the suppressed behavior can still be recovered while those features stay near their defended values.

SAE refusal steering and constrained recovery. SAE-based refusal steering identifies refusal-associated features and amplifies or suppresses them during inference [16, 23]. We use this setting as

a diagnostic: once refusal features are clamped, is non-refusal behavior actually eliminated? Inspired by AlphaSteer’s null-space perspective [18], we project single-layer recovery updates into the null space of the selected SAE encoder directions, and extend the same idea to cross-layer settings with the local feature-map Jacobian. Different from AlphaSteer, we adapt the null-space idea for recovery diagnostics: instead of preserving utility while steering, we use it to test whether behavior can recover without directly reactivating the targeted SAE features.

3 Preliminaries

Sparse autoencoders. Let M be a transformer language model and let $h_\ell(x) \in \mathbb{R}^{T \times d}$ denote the residual-stream activation at layer ℓ for an input sequence x . A sparse autoencoder (SAE) maps this activation to sparse latent features and reconstructs it as

$$z_\ell(x) = E_\ell(h_\ell(x)), \quad \hat{h}_\ell(x) = D_\ell(z_\ell(x)).$$

The coordinates of $z_\ell(x)$ are SAE features. For a selected feature set $\mathcal{S}$, we write $z_{\ell,\mathcal{S}}(x)$ for the corresponding feature activations.

Feature-level interventions. A feature-level intervention selects a feature set $\mathcal{S}$ and sets those features to defended values $c_\mathcal{S}$. Zero ablation corresponds to $c_\mathcal{S} = 0$, while refusal clamping may set selected refusal features to a nonzero defended value. Following standard SAE intervention practice, we preserve the SAE reconstruction residual and apply

$$h_\ell^{\text{def}}(x) = D_\ell(\text{clamp}_\mathcal{S}(z_\ell(x); c_\mathcal{S})) + (h_\ell(x) - \hat{h}_\ell(x)),$$

where $\text{clamp}_\mathcal{S}$ sets the selected SAE feature to $c_\mathcal{S}$ and leaves all other features unchanged. We denote the post-intervention residual by $h_\ell^{\text{def}}(x)$ and call it the *defended residual state*. All recovery experiments start from this state.

Valid flips. Let B be a task-specific predicate indicating whether the target behavior is present. We evaluate recovery only on *valid flips*: examples where the base model exhibits the target behavior but the active SAE intervention suppresses it. This conditioning ensures that recovery is measured only when there is a suppressed behavior to restore. The formal definition and task-specific instantiations are given in Appendix A.

Causal handles versus complete bottlenecks. A selected feature set $\mathcal{S}$ is a useful *causal handle* if intervening on it changes the target behavior. This is weaker than being a *complete intervention bottleneck*: once the clamp is active, no admissible residual perturbation should restore the suppressed behavior. Our experiments test this stronger condition.

4 Post-Intervention Recovery

We introduce *post-intervention recovery* to test if an intervention is a true *complete bottleneck* or just a bypassable *causal handle*. We frame this as a constrained optimization problem: finding a small perturbation to restore the suppressed behavior without altering the active SAE clamp. Figure 1 summarizes the method pipeline: the intervention first maps the original residual state to a defended state, and recovery then searches for a constrained residual perturbation that restores the suppressed behavior while keeping the clamp active.

Threat model. We study whether the model can recover its original behavior after the SAE intervention has been applied. Under a white-box setting, the optimizer can inspect the defended model, but it cannot change the model weights, remove the SAE clamp, or choose a different set of clamped features. All we allowed is to add an additive perturbation δ_x to the defended residual state:

$$h_\ell^{\text{rec}}(x) = h_\ell^{\text{def}}(x) + \delta_x.$$

We define δ_x as a *recovery path*. A successful recovery path would restore the behavior without reactivating the clamped feature itself. In other words, the clamp is still in place. The targeted SAE feature remains suppressed. But the model’s original behavior comes back by adding δ_x .

Finding such a path shows that the model does not rely on a single internal route for this behavior. Rather, the behavior can be recovered through alternative computational paths that bypass the defense.